# PACE (PBL Portal) - Technology Stack

PACE is the university platform for managing the complete Project-Based Learning lifecycle: teams, weekly progress reports, supervisor evaluation, final grading, announcements and queries. It ships as one web application with three portals (student, faculty/supervisor and admin) behind a single Supabase backend.

## Frontend

- **Next.js 16 (App Router)** The main frontend framework. One codebase serves every portal, with file-based routing, built-in data fetching and first-class TypeScript support. Chosen because it is the most productive framework for a data-driven dashboard app and deploys wherever the Vercel target expects.
- **React 19** The UI library under Next.js. All pages are client components, so the interface is fully interactive (forms, timelines, document viewer) while the data comes from Supabase.
- **TypeScript** End-to-end static typing. Domain types (Group, WeeklyReport, Grade, Ticket) are defined once and shared by every page and repository function, so most wiring mistakes are caught at compile time instead of in the browser.
- **Tailwind CSS v4** Utility-first styling, no separate CSS files to keep in sync. The whole visual language - cards, badges, progress bars, side panels - is composed from a small set of design tokens, which keeps the three portals consistent.
- **Component library (shadcn-style primitives)** Reusable UI primitives - Card, Button, Field, Badge, FileDrop, PageHeader - live in src/components/ui and are used by every page, so screens stay uniform and new pages are assembled fast.
- **Document previewing (docx-preview and a PDF viewer)** Students and supervisors open uploaded Word files and PDFs inside the portal, with their original layout, fonts and tables, without leaving the app.

# Backend - Supabase

All data, authentication and business rules live in Supabase, a hosted Postgres database with ready-made Auth and object storage. The frontend talks to it directly through the Supabase client; no custom API server is needed for the main features, which keeps the stack small.

## PostgreSQL

- **Where it is used** Every table of the system - profiles, students, faculty, groups and team members, weekly reports, milestones, grades, tickets, announcements, notifications - is a plain relational table. Postgres gives transactions, referential integrity and the ability to put logic inside the database.
- **Why it is used** One managed, highly available database; the schema is versioned in supabase/migrations so a fresh environment rebuilds itself; and it supports the next two features that make the app safe.

## Security definer SQL functions (RPCs)

- **Where it is used** The business logic is written as SQL functions the client calls by name: create_group, invite_member, respond_to_invitation, submit_milestone, save_group_grades, publish_group_grades, raiseTicket, raise_supervisor_change and the approval flows around it. A private app_private schema holds shared helpers like my_student_id() and require_leader().
- **Why it is used** Client code can only read and call RPCs; it never writes rows directly. Rules such as only the team lead may upload milestones, two supervisors must approve a supervisor change, and publishing grades is refused until every group is graded are enforced by the database itself, so no client bug or tampered request can break them. Everything happens in one atomic transaction on the server.

## Row Level Security

- **Where it is used** Every table has RLS policies: a student sees only their own group's rows, a supervisor sees only the groups they supervise, an admin sees all of them, and attachments follow the same rules.
- **Why it is used** Filtering happens in Postgres, not in the frontend. Even a crafted query from the browser cannot reach another team's reports, grades or files, which is the right place for confidentiality in a multi-tenant system.

## Supabase Auth

- **Where it is used** Sign in and sign up with university email and password, profiles linked to the auth user, plus email-OTP password recovery and password changes.
- **Why it is used** A hosted, audited authentication service removes the whole class of home-grown login bugs; roles (student, faculty, admin) are kept in the profiles table next to the verified account.

# Deployment and Document Storage

## Deployed on Vercel

- **Where it is used** The whole Next.js application, including the small number of API routes (currently the central LinkedIn account posting), is deployed on Vercel at pbl-sem-5-sigma.vercel.app.
- **Why it is used** Vercel is the platform built for Next.js: pushing to GitHub automatically builds, previews and deploys the app with HTTPS, environment variables (Supabase URLs and keys) and edge-network delivery, so there is no server to run, patch or scale.

## Documents in Supabase Storage (object storage)

- **Where it is used** Every uploaded file - signed PBL forms, weekly report attachments, midterm and endterm PPTs, synopses, query attachments, announcement files, achievement proofs and profile photos - is stored as an object in Supabase Storage, in private buckets (submissions, announcements, achievements) and a public avatars bucket. Files live under a folder per group, for example submissions/<group id>/report-<week id>/.
- **How access works** Uploads go straight from the browser to Storage. Downloads and in-app viewing use signed URLs that expire after ten minutes, so links in old pages never keep working; the viewer fetches the file and renders PDFs and Word documents on the page.
- **Why it is used** Object storage is made for exactly this workload: cheap, durable, CDN-delivered files that scale without touching the database. Keeping the bytes out of Postgres keeps the database small, and private buckets plus signed URLs mean no student work is ever publicly reachable.

## Summary

In short: a Next.js 16 and React 19 frontend, typed end to end and styled with Tailwind v4; Supabase as the entire backend - Postgres with security definer functions and Row Level Security for data and rules, Auth for accounts, and object storage for every document; and Vercel as the one-click deployment target. Each choice was made to keep the system simple, safe by default and cheap to run.